

`useRef` in React

`useRef` is a React Hook used to:

1. Access DOM elements directly
 2. Store mutable values without re-rendering
 3. Persist values between renders
-

Syntax

```
const ref = useRef(initialValue);
```

What `useRef` Returns

`useRef()` returns an object:

```
{
  current: initialValue
}
```

Example:

```
const inputRef = useRef(null);
```

Creates:

```
{
  current: null
}
```

Important Feature

Changing `ref.current`:

- ☐ DOES NOT re-render component

Unlike `useState`.

Basic Uses of `useRef`

Use Case	Purpose
DOM Access	Access input/button/div
Store Previous Value	Persist data
Timer IDs	Store interval/timeout
Avoid Re-render	Mutable storage
Focus Inputs	Auto focus
Video Controls	Play/Pause videos

Example 1 — Accessing Input Field

Goal

Click button → focus input automatically.

Code

```
import React, { useRef } from 'react';

function App() {

  const inputRef = useRef(null);

  const focusInput = () => {

    inputRef.current.focus();
  };

  return (

    <div>

      <input
        type="text"
        ref={inputRef}
      />

    </div>

  );
}
```

```
        <button onClick={focusInput}>
          Focus Input
        </button>

      </div>

    );
  }

export default App;
```

How This Works

Step 1

```
const inputRef = useRef(null);
```

Creates ref object:

```
{
  current: null
}
```

Step 2

```
ref={inputRef}
```

React attaches DOM element to:

```
inputRef.current
```

Now:

```
inputRef.current = <input />
```

Step 3

```
inputRef.current.focus();
```

Directly focuses input.

Output

Button click automatically focuses text box.

Visual Flow

```
useRef()  
  ↓  
Attach to DOM using ref  
  ↓  
Access DOM using ref.current
```

Example 2 — Store Previous Value

Very common interview question.

Goal

Show previous counter value.

Code

```
import React, { useState, useEffect, useRef } from 'react';

function App() {

  const [count, setCount] = useState(0);

  const previousCount = useRef(0);

  useEffect(() => {

    previousCount.current = count;

  }, [count]);

  return (

    <div>

      <h1>Current: {count}</h1>

      <h2>Previous: {previousCount.current}</h2>

      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>

    </div>

  );
}

export default App;
```

How This Works

Initial Render

```
count = 0
previousCount.current = 0
```

After Increment

```
count = 1
```

Before next render:

```
previousCount.current = old count
```

Now UI shows:

```
Current: 1  
Previous: 0
```

Why `useRef` Works Here

Because refs persist values between renders without causing re-render.

Example 3 — Stopwatch / Timer

Goal

Store interval ID.

Code

```
import React, { useState, useRef } from 'react';

function Timer() {

  const [count, setCount] = useState(0);

  const intervalRef = useRef(null);

  const startTimer = () => {

    intervalRef.current = setInterval(() => {

      setCount(prev => prev + 1);

    }, 1000);
  };

  const stopTimer = () => {

    clearInterval(intervalRef.current);
  };

  return (

    <div>

      <h1>{count}</h1>

      <button onClick={startTimer}>
        Start
      </button>

      <button onClick={stopTimer}>
        Stop
      </button>

    </div>

  );
}

export default Timer;
```

Why Use Ref Here?

If interval ID stored in variable:

```
let interval;
```

Value resets on re-render.

But `useRef` persists value.

`useRef` VS `useState`

This is VERY important.

`useState`

```
const [count, setCount] = useState(0);
```

Updating state:

```
setCount(1);
```

☐ Causes re-render

`useRef`

```
const countRef = useRef(0);
```

Updating ref:

```
countRef.current = 1;
```

☐ Does NOT re-render

Difference Table

Feature	useState	useRef
Causes re-render	Yes	No
Stores mutable data	Yes	Yes
Persists between renders	Yes	Yes
Used for UI updates	Yes	No
DOM access	No	Yes

Example 4 — Count Renders

Code

```
import React, { useState, useEffect, useRef } from 'react';

function App() {

  const [value, setValue] = useState("");

  const renderCount = useRef(0);

  useEffect(() => {

    renderCount.current += 1;

  });

  return (

    <div>

      <input
        value={value}
        onChange={(e) => setValue(e.target.value)}
      />

      <h2>Render Count: {renderCount.current}</h2>

    </div>

  );
}

export default App;
```

Output

Every typing increases render count.

Useful for debugging.

Example 5 — Auto Focus on Page Load

Code

```
import React, { useEffect, useRef } from 'react';

function App() {

  const inputRef = useRef();

  useEffect(() => {

    inputRef.current.focus();

  }, []);

  return (
    <input ref={inputRef} />
  );
}

export default App;
```

Real-World Use Cases

`useRef` commonly used in:

Scenario	Example
Forms	Focus input
Media	Video/audio controls
Animations	DOM manipulation
Timers	Store interval ID
Previous values	Old state tracking
Performance	Avoid re-renders

Forwarding Refs

Sometimes parent wants access to child DOM.

React provides:

`forwardRef()`

Advanced topic.

Example

```
const Input = forwardRef((props, ref) => {  
  return <input ref={ref} />;  
});
```

Important Rules

1. Updating Ref Does NOT Re-render

Wrong expectation:

```
ref.current = 100;
```

UI will NOT update automatically.

2. Ref Value Persists

Even after re-renders:

```
ref.current
```

stays same.

3. Avoid Overusing DOM Manipulation

React prefers declarative UI.

Use refs only when necessary.

Common Mistakes

Mistake 1 — Expecting Re-render

☐ Wrong

```
ref.current++;
```

UI won't update.

Mistake 2 — Using Ref Instead of State

Wrong usage:

```
const nameRef = useRef("");
```

for form UI updates.

Use `useState` instead.

When to Use `useRef`

Use when:

- ☐ Need DOM access
 - ☐ Need persistent mutable value
 - ☐ Need timer IDs
 - ☐ Need previous values
 - ☐ Want to avoid re-renders
-

When NOT to Use `useRef`

Avoid when:

- ☐ UI should update on change
- ☐ State affects rendering

Then use `useState`.

Mental Model

Think of `useRef` like:

A storage box that survives renders

React re-renders component:

But box remains same

Simple Analogy

Imagine classroom attendance register.

`useState`

Teacher updates board.

Everyone sees update immediately.

`useRef`

Teacher writes private note in notebook.

Classroom UI does not change.

Final Summary

`useRef`:

- ☐ Accesses DOM elements
 - ☐ Stores mutable values
 - ☐ Persists across renders
 - ☐ Does NOT trigger re-render
 - ☐ Great for timers, focus, previous values
-

Most Important Interview Question

Difference Between `useRef` and `useState`

`useState`

- Stores state
- Causes re-render on update
- Used for UI rendering

`useRef`

- Stores mutable value
- Does NOT cause re-render
- Used for DOM access and persistent values